Junxi Lin
Github: Junxi-Lin

Assignment 6 - Device Driver

Description:

This Assignment is to write a Linux device driver in C language, which can be loaded and executed in the Linux kernel. The device driver must support interaction with users through standard file operations (such as opening, reading, writing, and ioctl). The program uses the Caesar cipher to perform text encryption and decryption operations. Users can pass a string to the device driver, and the driver will return the encrypted or decrypted version of the string. Additionally, the encryption key can be dynamically set through the ioctl command.

Approach:

Firstly, my focus was on establishing the basic structure of the device driver. I implemented the required functions such as opening, release, reading, writing, and ioctl. I used a structure to store the data of the device, including the text buffer, text size, encryption key, and a flag indicating whether the text is encrypted. When the device is opened, memory is allocated using vmalloc, and when it is closed, the memory is released.
Next, I dedicated myself to implementing the data interaction with the device. In the write function, copy_from_user is used to copy the user input into the driver, and in the read function, copy_to_user is used to return the processed result to the user program. Initially, I encountered some problems in this aspect because I did not handle the buffer correctly, but after adjusting the size and ensuring that the string ended with a null character, it worked properly.
After that, I implemented the Caesar cipher logic. I did not use a fixed shift value, but decided to allow the user to dynamically set the key. So, I added a command named "k" for setting the key, and used "e" and "d" to control the encryption and decryption operations. Inside the ioctl function, I traversed each character in the string and performed shift processing based on the key. For encryption operations, I added the key, while for decryption operations, I performed subtraction.
Then I created a test program. Initially, I wrote this program using command-line parameters, but later realized that the program should allow the user to input values during execution, so I modified it to be interactive. The test program requires the user to input a string, a key, and

choose whether to encrypt or decrypt. Then it opens /dev/secret, writes the string to the driver, sends the ioctl command to set the key and select the operation, and finally reads the result and returns it.
To test everything, I used some simple examples, such as using a key of 5 to encrypt "hello" to "mjqqt", and then decrypting it back to "hello". This helps confirm that the driver and the test program can work together properly.

Issues and Resolutions

One issue that I must pay special attention to is the correct usage of the "copy_to_user" and "copy_from_user" functions. Since the device driver runs in the kernel space while the test program runs in the user space, I cannot directly access the data. Therefore, I have to use these functions to transfer the data. Initially, I found that the output of the driver was not always accurate. This was mainly because the buffers were not handled properly. Sometimes too many bytes were copied, and sometimes the strings did not end with a null character correctly, which resulted in additional garbage characters appearing in the output. To solve this problem, I ensured that the number of bytes copied to the buffer was limited within a certain range and always left space for the null character terminator. After copying the data from the user space, I explicitly added "\0" at the end of the string. This ensured the correct format of the data and enabled the read operation to return the expected encryption or decryption result.

Anaysis: N/A

Screen shots of compilation:

Device driver:

```
student@student:~$ cd csc415-assignment-6-device-driver-Junxi-Lin/Module
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ make clean
make -C /lib/modules/`uname -r`/build M=/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module clean
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-23-generic'
make[2]: Entering directory '/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module'
  CLEAN   Module.symvers
make[2]: Leaving directory '/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-23-generic'
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ make
make -C /lib/modules/`uname -r`/build M=/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-23-generic'
make[2]: Entering directory '/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: aarch64-linux-gnu-gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0
  You are using:           gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0
  CC [M]  secret.o
  MODPOST Module.symvers
  CC [M]  secret.mod.o
  CC [M]  .module-common.o
  LD [M]  secret.ko
  BTF [M] secret.ko
Skipping BTF generation for secret.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/student/csc415-assignment-6-device-driver-Junxi-Lin/Module'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-23-generic'
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ ls
Makefile  Module.symvers  modules.order  secret.c  secret.ko  secret.mod  secret.mod.c  secret.mod.o  secret.o
```

Driver Load:

```
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ sudo rmmod secret
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ sudo insmod secret.ko
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ sudo rm /dev/secret
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ sudo mknod /dev/secret c 415 0
```

Screen shots of the execution of the program:

```
[sudo: authenticate] Password:
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Module$ cd ../Test
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Test$ ./Lin_Junxi_HW6_main
Secret Device Driver Test Program
---------------------------------
Enter text: hello
Enter Caesar cipher key: 5

Choose an option:
1. Encrypt
2. Decrypt
Enter choice: 1

Result: mjqqt
student@student:~/csc415-assignment-6-device-driver-Junxi-Lin/Test$ ./Lin_Junxi_HW6_main
Secret Device Driver Test Program
---------------------------------
Enter text: mjqqt
Enter Caesar cipher key: 5

Choose an option:
1. Encrypt
2. Decrypt
Enter choice: 2

Result: hello
```